# The Gammacount-likelihood

## Parametrisation

The Gammacount-distribution is a discrete probability distribution on $0, 1, 2, 3, \ldots$, where

$$\text{Prob}(y) = G(y\alpha, \beta) - G((y+1)\alpha, \beta)$$

where

$$G(a, b) = \frac{1}{\Gamma(a)} \int_0^b x^{a-1} \exp(-x) dx.$$

The reciprocal of the expected waiting time depends on the linear predictor

$$(\alpha/\beta)^{-1} = E \exp(\eta),$$

so that high values of $\eta$ corresponds to high values of $y$, and low values of $\eta$ corresponds to low values of $y$, and $\log(E)$ is an offset. In the case where $\alpha = 1$, then we are back to the Poisson.

## Link-function

The linear predictor $\eta$ is linked to the reciprocal of the expected waiting time, by a log-link,

$$(\alpha/\beta)^{-1} = E \exp(\eta), \qquad E > 0.$$

## Hyperparameter

The hyperparameter is the parameter $\alpha$, which is represented as

$$\alpha = \exp(\theta)$$

and the prior is defined on $\theta$.

## Specification

- `family="gammacount"`
- Required arguments: $y$ (and $E$, default one)

### Hyperparameter spesification and default values

**doc** `A Gamma generalisation of the Poisson likelihood`

**hyper**

**theta**

**hyperid** `59001`
**name** `log alpha`
**short.name** `alpha`
**output.name** `Log-alpha parameter for Gammacount observations`
**output.name.intern** `Alpha parameter for Gammacount observations`
**initial** `0`
**fixed** `FALSE`
**prior** `pc.gammacount`
**param** `3`

**to.theta** `function(x) log(x)`
**from.theta** `function(x) exp(x)`

**survival** `FALSE`

**discrete** `FALSE`

**link** `default log`

**pdf** `gammacount`

## Example

In the following example we estimate the parameters in a simulated example.

```
G = function(Alpha, Beta) {
    return (pgamma(Beta, shape=Alpha, rate=1))
}

n = 1000
x = rnorm(n)
eta = 1 + x
alpha = 1.5
T = 1
m = 100
y = numeric(n)
prob = numeric(m+1)

for(i in 1:n) {

    ## compute the discrete probability distribution and
    ## then sample from it
    for(j in 1:m) {
        yy = j-1
        beta = alpha * exp(eta[i])
        prob[j] = (G(yy*alpha, beta*T) -
                   G((yy+1)*alpha, beta*T))
    }
    y[i] = sample(0:m, size=1, prob = prob)
}

r = (inla(y ~ 1 + x,
          data = data.frame(y, x),
          family = "gammacount"))
summary(r)
```

## Notes

None.